TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE VERACRUZ

Despliegue de MongoDB con persistencia de datos usando Podman

Leo Pantoja Alberto Francisco

22020054

Ing. José Nahum Prieto Velázquez

BD NOSQL

23 de Septiembre del 2026



Objetivos

- Desplegar un contenedor de MongoDB utilizando **Podman**.
- Configurar un volumen montado desde el sistema host para garantizar la **persistencia de datos** (evitando la pérdida de información al reiniciar o detener el contenedor).
- Verificar el correcto almacenamiento y recuperación de la información almacenada en la base de datos.

1. Preparación del Directorio de Persistencia

Se creó un directorio local en el directorio personal del host para almacenar la información de la base de datos MongoDB:

```
tito@fsociety:~$ mkdir -p ~/mongo_data
```

mkdir -p ~/mongo_data

- **Explicación:** El flag -p asegura que el directorio se cree si no existe. Este directorio servirá para mapear el volumen hacia el contenedor.

2. Despliegue del Contenedor de MongoDB

Se ejecutó el contenedor de MongoDB montando el volumen local y mapeando el puerto correspondiente:

```
tito@fsociety:~$ podman run -d --name mongo-db -v ~/mongo_data:/data/db:Z -p 27017:27017 docker.io/library/mongo:latest
Trying to pull docker.io/library/mongo:latest...
Getting image source signatures
Copying blob edd1ed89f0d4 done   |
Copying blob 452c7dd3efc2 done   |
Copying blob 629b21fcd7ff done   |
Copying blob 6924226aa25f done   |
Copying blob dfc8c140f336 done   |
Copying blob bbd0b49c84d6 done   |
Copying blob d05cf6da107e done   |
Copying blob 00c2e1442598 done   |
Copying config 4d9c509681 done   |
Writing manifest to image destination
b37bb15c6ccac2c13f31cc19a7dfa657413ee3029c8411a600b6d793643ea1f0
tito@fsociety:~$
```

podman run -d --name mongo-db -v ~/mongo_data:/data/db:Z -p 27017:27017 docker.io/library/mongo:latest

- **Desglose de parámetros empleados:**
  - -d: Ejecuta el contenedor en segundo plano (*detached mode*).
  - --name mongo-db: Asigna el nombre mongo-db al contenedor.
  - -v ~/mongo_data:/data/db:Z: Asocia la carpeta local ~/mongo_data con la ruta /data/db dentro del contenedor. La opción :Z asigna las etiquetas SELinux necesarias para permitir el acceso correcto desde Podman.
  - -p 27017:27017: Mapea el puerto 27017 del host al puerto 27017 del contenedor.

- docker.io/library/mongo:latest: Descarga e instala la última imagen oficial de MongoDB desde Docker Hub.

3. Inserción de Datos Iniciales

Se ingresó a la consola interactiva de MongoDB dentro del contenedor para insertar información de prueba:

```
tito@fsociety:~$ podman exec -it mongo-db mongo
MongoDB shell version v4.4.30
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("64336371-eca2-4b62-a41f-11cfbd97acc2") }
MongoDB server version: 4.4.30
---
The server generated these startup warnings when booting:
        2026-09-24T02:40:00.444+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http:/
/dochub.mongodb.org/core/prodnotes-filesystem
        2026-09-24T02:40:03.500+00:00: Access control is not enabled for the database. Read and write access to data and configuration
 is unrestricted
        2026-09-24T02:40:03.500+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never'
---
> use mi_base
switched to db mi_base
> db.prueba.insertOne({ nota: "Dato guardado antes de apagar" })
{
        "acknowledged" : true,
        "insertedId" : ObjectId("6ab48f0265e484da372bcd24")
}
> exit
bye
tito@fsociety:~$ podman stop mongo-db
```

podman exec -it mongo-db mongo
Dentro de la shell de MongoDB, se realizaron los siguientes comandos:

// Seleccionar/Crear la base de datos
use mi_base

// Insertar un registro en la colección "alumnos"
db.prueba.insertOne({ nota: “Dato guardado antes de apagar" })

// Consultar el registro guardado
db.alumnos.find()

- **Resultado:** La inserción devolvió acknowledged: true con un insertedId asignado (ObjectId("6ab48c71396718293647bf")), confirmando que la información quedó registrada en la colección de prueba.

4. Prueba de Persistencia de Datos

Posteriormente, tras reiniciar o apagar el servicio/contenedor, se volvió a acceder a la base de datos para verificar que los registros guardados previamente continuaran intactos en el volumen persistente:

```
tito@fsociety:~$ podman stop mongo-db
mongo-db
tito@fsociety:~$ podman start mongo-db
mongo-db
```



podman exec -it mongo-db mongo
Dentro de la shell de MongoDB:

JavaScript
use mi_base
db.prueba.find()

- **Resultado:** El comando db.prueba.find() o la consulta a las colecciones creadas arrojó los registros previamente guardados (ej. {"nota": "Dato guardado antes de apagar"}), demostrando que los datos sobrevivieron al ciclo de vida del contenedor.

Conclusión

El uso de volúmenes con Podman (empleando -v ~/mongo_data:/data/db:Z) asegura que toda la información generada por MongoDB se guarde directamente en el sistema de archivos del host. De este modo, los datos persisten aunque el contenedor sea detenido, eliminado o reiniciado.